

satsim — código explicado

Simulador de posicionamento de satélites

Documento vivo · iniciado em 02/10/2026 · branch `semana-1`

Sobre este documento. Este arquivo contém a explicação detalhada de alguns scripts do projeto satsim. Ele complementa as docstrings e os comentários do código: lá fica o *o quê* de cada função; aqui fica o *porquê* das decisões, os conceitos de Python e de mecânica orbital envolvidos e como cada peça se encaixa no restante do simulador. As explicações são acrescentadas à medida que os scripts são revisados, uma seção por script.

Convenções gerais (unidades SI, ângulos em radianos, tempo em UTC, referencial ECI) estão no ADR docs/adr/0001-arquitetura.md.

Scripts explicados

1. `src/satsim/propagators/base.py` — interface base dos propagadores

1. `base.py` — a interface dos propagadores

`src/satsim/propagators/base.py` · card S1-05

O `base.py` não calcula nada: ele define o **contrato** que todo propagador do projeto precisa cumprir. O conteúdo é curto:

```
class Propagator(ABC):
    """Propagador de órbita: estado ECI em função do tempo desde a época."""

    @property
    @abstractmethod
    def epoch(self) -> datetime:
        """Época do propagador (UTC, tz-aware): o instante t = 0."""

    @abstractmethod
    def propagate(self, t: ArrayLike) -> tuple[NDArray[np.float64], NDArray[np.float64]]:
        """t: segundos desde ``epoch``, shape () ou (N,).

        Retorna (r, v) em ECI, SI [m, m/s], sempre com shape (N, 3).
        """

    def as_time_array(t: ArrayLike) -> NDArray[np.float64]:
        arr = np.atleast_1d(np.asarray(t, dtype=float))
        if arr.ndim != 1:
            raise ValueError(f"t deve ser escalar ou 1D, recebido shape {arr.shape}")
        return arr
```

A classe abstrata `Propagator(ABC)`

`ABC` (*Abstract Base Class*, do módulo `abc` da biblioteca padrão) é uma classe que **não pode ser instanciada diretamente**. Ela funciona como um molde: declara *quais* métodos as

subclasses devem ter, sem dizer *como* eles funcionam.

No satsim, ela materializa a regra central do ADR: **todo propagador implementa `propagate(t) → (r, v)`**. Ao longo do projeto haverá vários propagadores:

Propagador	Semana	Como calcula
KeplerPropagator	1	dois corpos, analítico
J2 médio	2	elementos com deriva secular
Cowell	3	integração numérica com J2 e arrasto
SGP4	3	a partir de TLE (ISS real)

O resto do código (conversão ECI → ECEF, visibilidade, constelação, exportação para a GUI) depende só de `Propagator`, nunca de um propagador específico. Isso garante duas coisas:

- **Troca sem reescrita:** `Satellite(propagator=KeplerPropagator(...))` vira `Satellite(propagator=CowellPropagator(...))` e nada mais muda.
- **Erro cedo:** se alguém criar um propagador e esquecer `propagate` ou `epoch`, o Python recusa a criação do objeto, em vez de falhar no meio de uma simulação:

```
TypeError: Can't instantiate abstract class MeuProp without an implementation  
for abstract method 'propagate'
```

Por isso os métodos na ABC só têm docstring, sem implementação: a docstring é a **especificação do contrato** (unidades, referencial e shapes).

O decorator `@abstractmethod`

Marca o método como **obrigatório**. No momento de instanciar, a ABC verifica se a subclasse sobrescreveu todos os métodos marcados. Sem esse decorator, a ABC não impediria nada: uma subclasse incompleta funcionaria até a primeira chamada e só então falharia.

O decorator `@property`

Transforma um método em um **atributo de leitura**: escreve-se `prop.epoch`, sem parênteses, em vez de `prop.epoch()`. Ele foi usado porque a época é um *dado* do propagador, não uma ação. Além disso, impede a atribuição acidental (`prop.epoch = outra_data` gera erro), o que protege a consistência: todos os `t` são medidos a partir dessa época, e mudá-la depois invalidaria os resultados.

Os dois decorators juntos em `epoch`

Significam que toda subclasse é obrigada a fornecer `epoch` como *property* de leitura. A ordem importa: `@property` por fora (em cima) e `@abstractmethod` por dentro. A ordem inversa não funciona corretamente no Python moderno. `propagate`, por sua vez, leva só `@abstractmethod`, porque é uma ação que recebe o argumento `t`.

A função `as_time_array`

Não é decorator nem faz parte da ABC: é um *helper* que todo propagador chama no início de `propagate`. Ele normaliza qualquer entrada de tempo (`5.0`, `[1, 2]`, `np.array(...)`) para um array 1D de floats com shape `(N,)`, rejeitando entradas com mais de uma dimensão. Assim, cada propagador não precisa repetir esse tratamento, e o contrato “sempre devolve `(N, 3)`” fica fácil de cumprir.